

bok25

基
礎
班

、**考研数据結構**



串 (字符串) 的定义

$$s = "a_1a_2a_3...a_n" \quad (n > 0)$$

零个或多个字符组成的有限序列

s: 串的名

"a₁a₂a₃...a_n": 串的值

a_i: 字符 (字母、数字.....)

n: 串的长度



串的模式匹配算法

1.BF算法 (Brute-Force, 暴力)

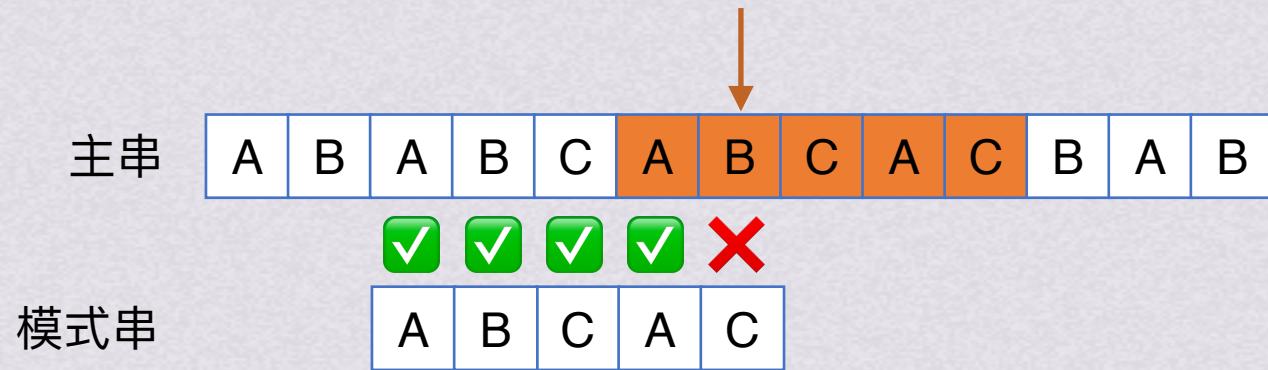
2.KMP算法



串的模式匹配算法

1.BF算法 (Brute-Force, 暴力)

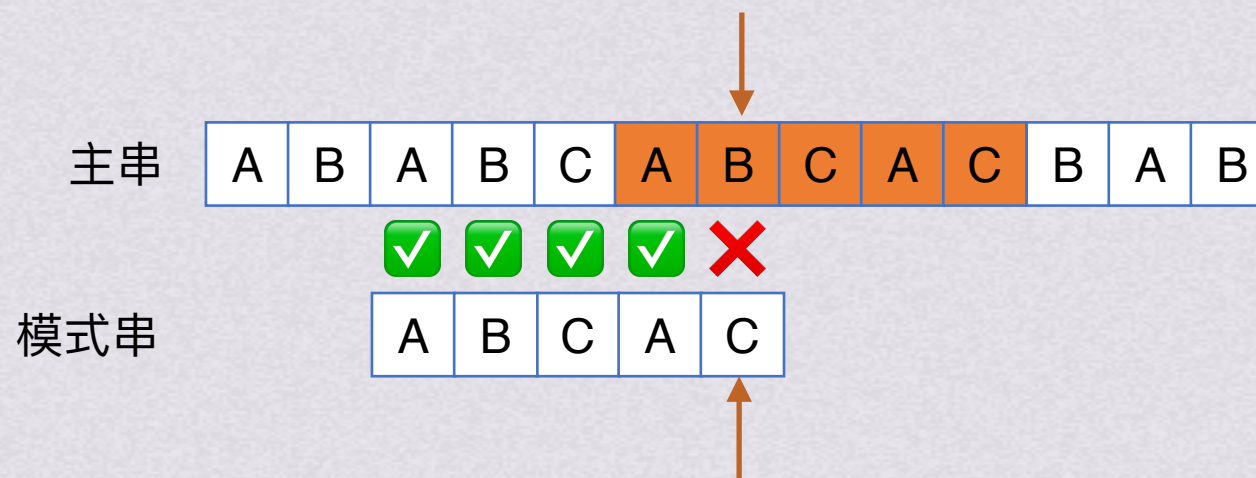
暴力算法的问题 笨！



比较到这一步时，瞪眼法就可以看出，一旦右移一次必然第一个比较就出错
把A对应上才有可能比较成功



串的模式匹配算法2.KMP算法



寻找模式串中失配位置之前的子串的最长相等前后缀



串的模式匹配算法2.KMP算法

模式串

A	B	C	A	C
---	---	---	---	---

ABCA的前缀：



A AB ABC



串的模式匹配算法2.KMP算法

模式串

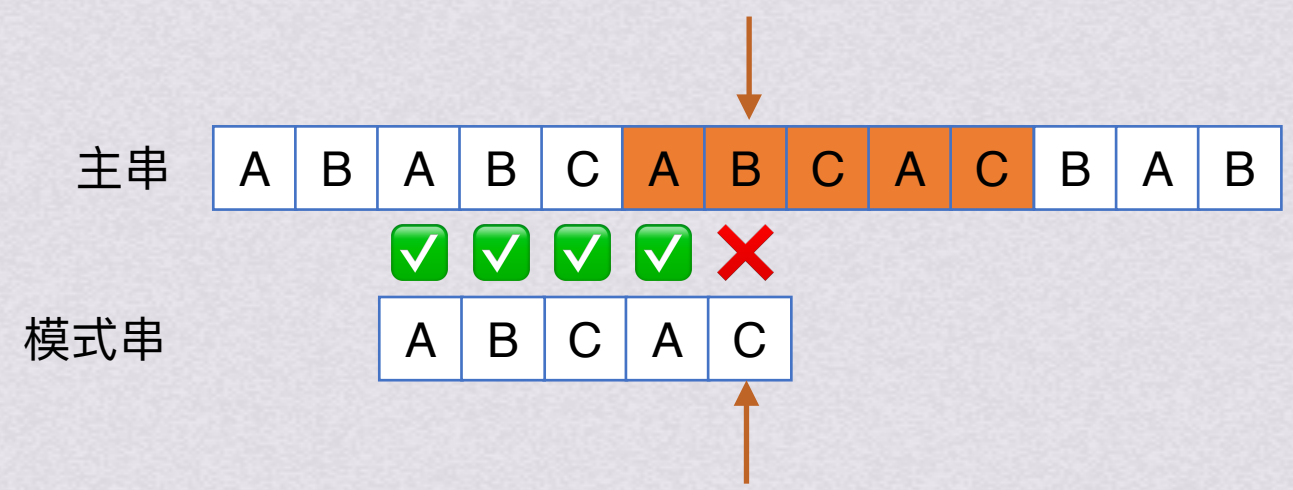
A	B	C	A	C
---	---	---	---	---

BCA CA A

→ ABCA的后缀

ABCA的前缀: A AB ABC

串的模式匹配算法2.KMP算法



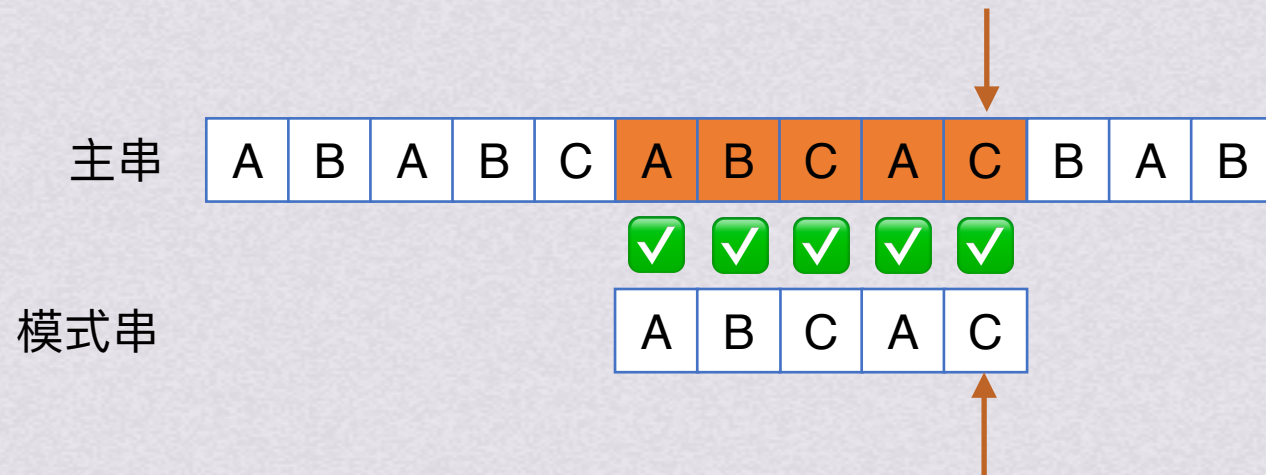
最长相等前后缀

ABCA的前缀:	A	AB	ABC
ABCA的后缀:	A	CA	BCA

接下来要做的操作是，右移模式串，保证指针左侧的模式串长度为最长相等前后缀的长度



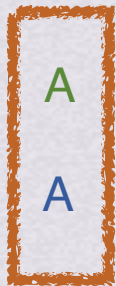
串的模式匹配算法2.KMP算法



最长相等前后缀

ABCA的前缀: A AB ABC

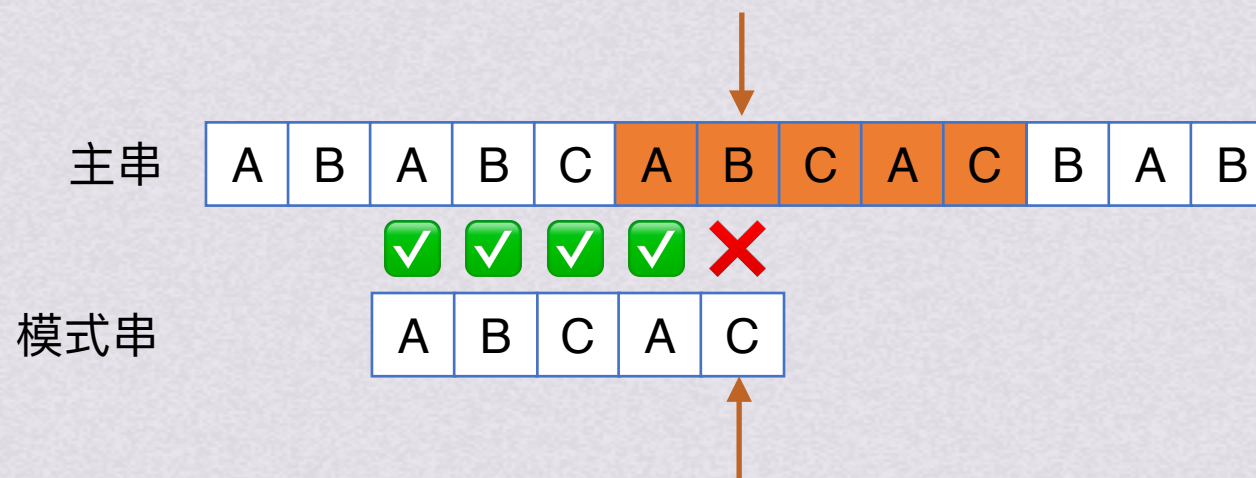
ABCA的后缀: A CA BCA



接下来要做的操作是，右移模式串，保证指针左侧的模式串长度为最长相等前后缀的长度



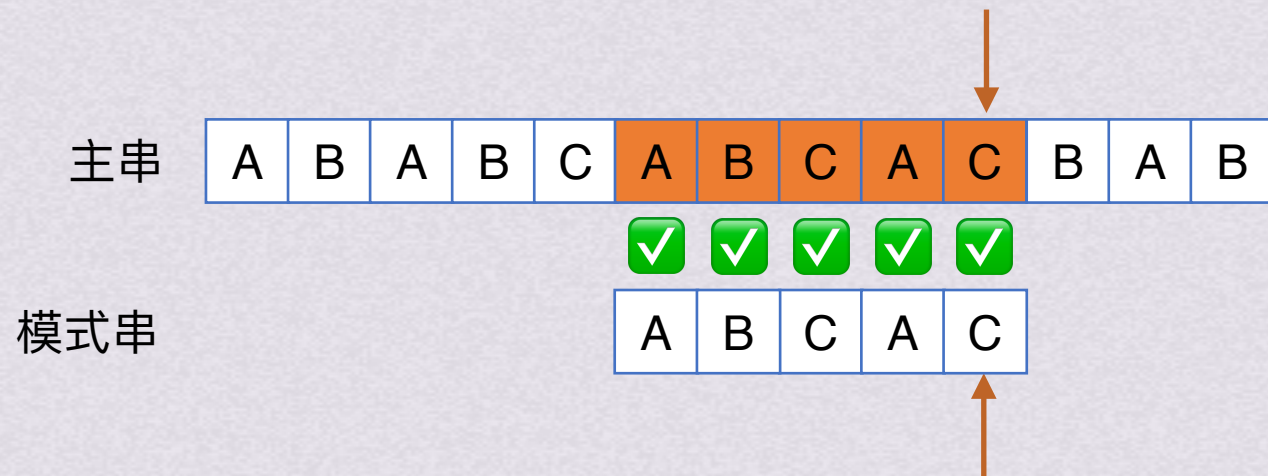
串的模式匹配算法2.KMP算法



接下来要做的操作是，右移模式串，保证指针左侧的模式串长度为最长相等前后缀的长度



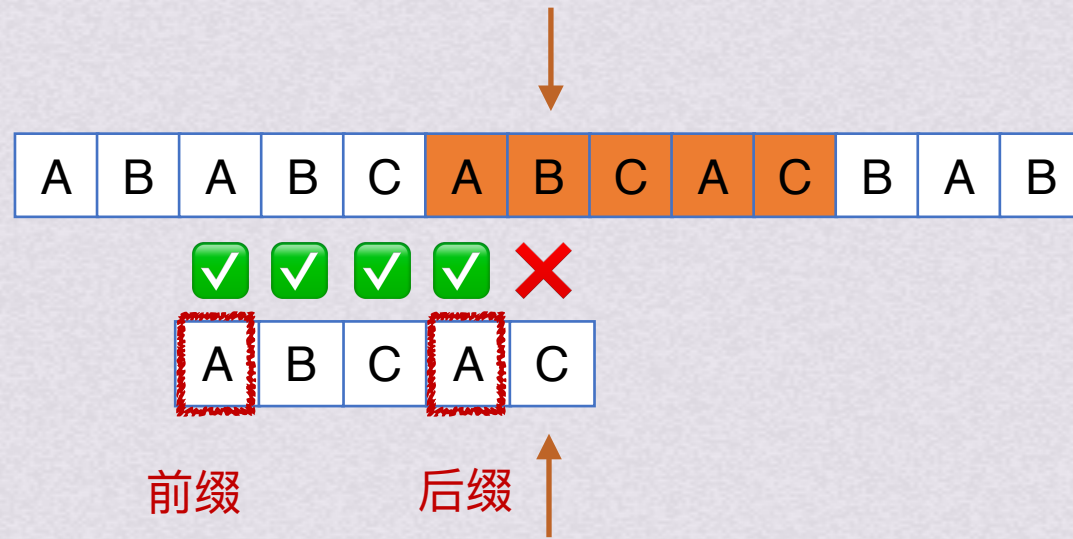
串的模式匹配算法2.KMP算法



接下来要做的操作是，右移模式串，保证指针左侧的模式串长度为最长相等前后缀的长度



串的模式匹配算法2.KMP算法

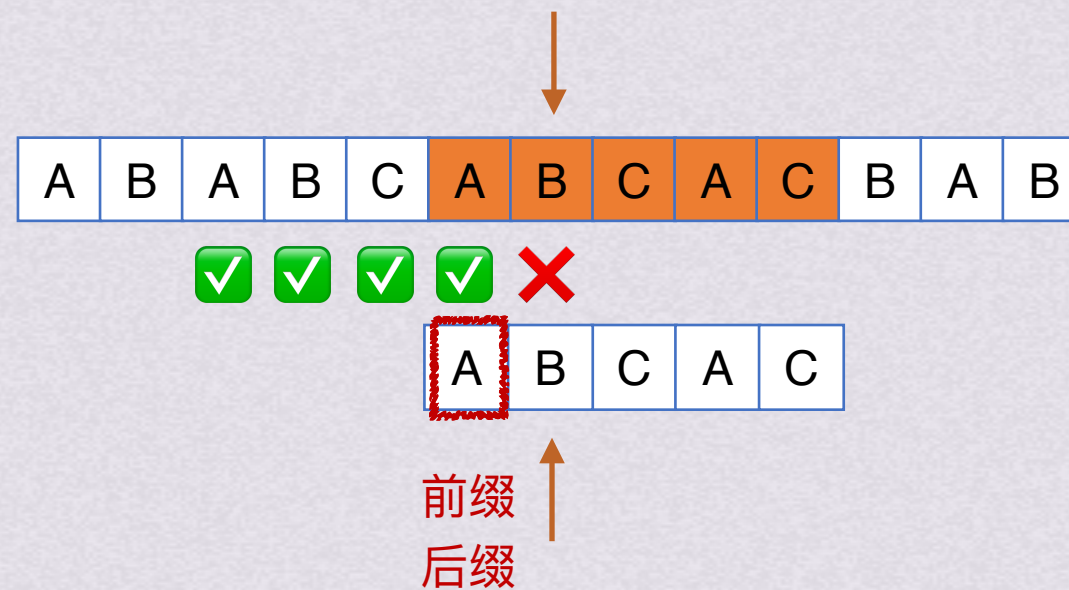


依据ABCA的最长相等前后缀移动

前缀移动到原本后缀的位置



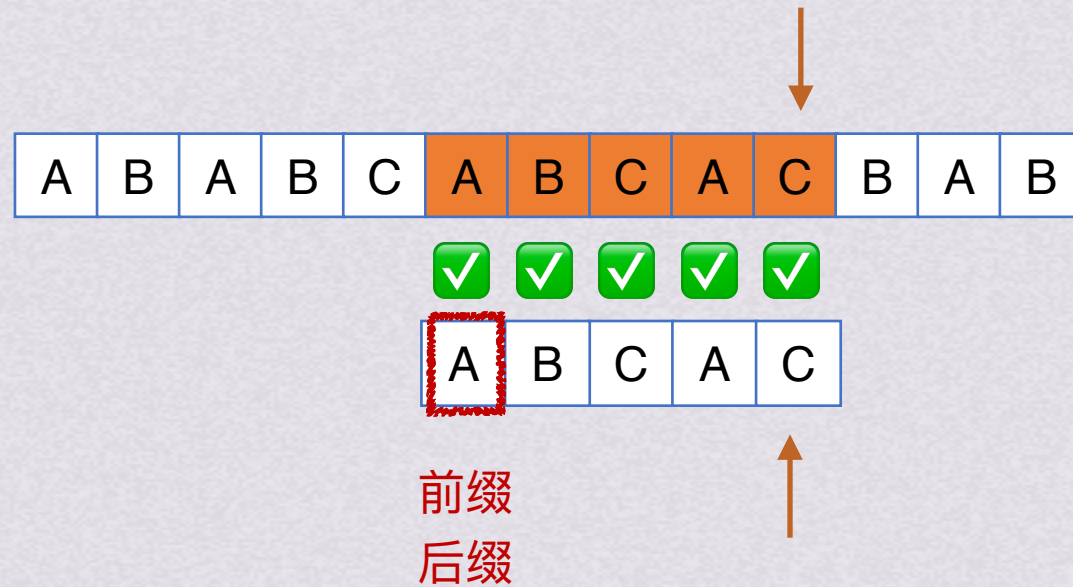
串的模式匹配算法2.KMP算法



依据ABCA的最长相等前后缀移动
前缀移动到原本后缀的位置



串的模式匹配算法2.KMP算法



依据ABCA的最长相等前后缀移动
前缀移动到原本后缀的位置



串的模式匹配算法2.KMP算法

抽象化的步骤：

- 1.设置指针，发现模式串某位失配
- 2.指针保持不动，寻找模式串在指针之前的部分，寻找该子串的最长相等前后缀
- 3.将前缀移动到原本后缀的位置，即保证指针左侧的模式串长度为最长相等前后缀的长度
- 4.移动完毕后，指针即可继续扫描进行新一轮的匹配



*实际内存中两个串均不移动，靠的是指针的移动来进行比较



串的模式匹配算法2.KMP算法

next数组

当模式串中的第*i*位发生失配时，模式串的第next[i]位将重新与主串当前位置进行比较

观察字符串信息 → 寻找最长相等前后缀 → 将前缀移动到原本后缀的位置



串的模式匹配算法2.KMP算法

当模式串中的第 i 位发生失配时，模式串的第 $next[i]$ 位将重新与主串当前位置进行比较

观察字符串信息 → 寻找最长相等前后缀 → $next$ 数组 → 将前缀移动到原本后缀的位置



串的模式匹配算法

2.KMP算法 `next`数组

模式串：

A	B	C	A	C
---	---	---	---	---



串的模式匹配算法

2.KMP算法 next数组

模式串编号: 1 2 3 4 5

模式串:

A	B	C	A	C
---	---	---	---	---

next数组: 0 1 1 1 2

↑

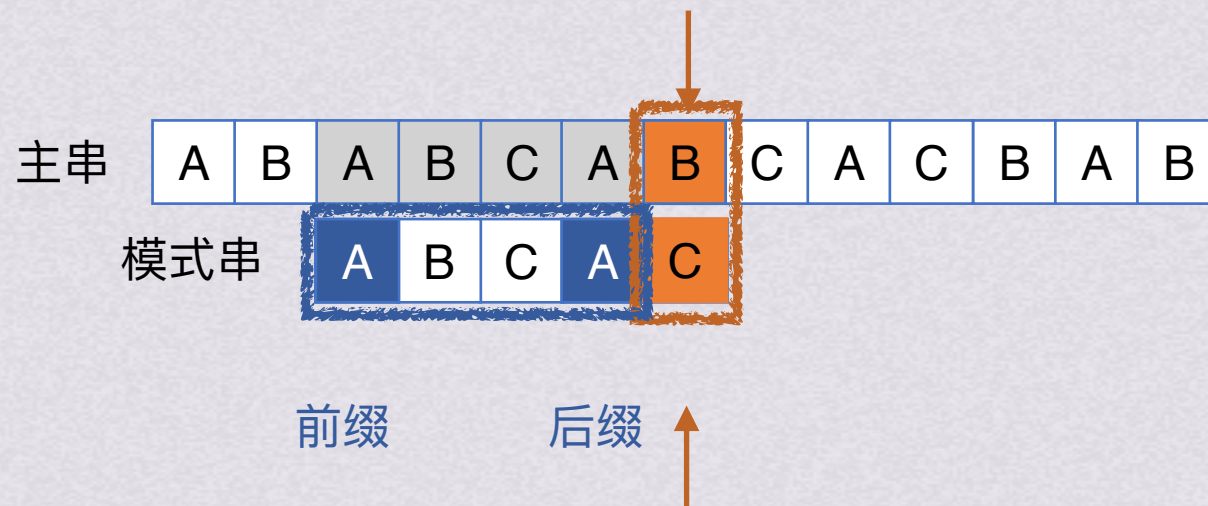
*注: 假设next数组从0开始, 模式串编号从1开始

小技巧: 在注*条件下, 此时next数组前两位可以直接写0,1



串的模式匹配算法

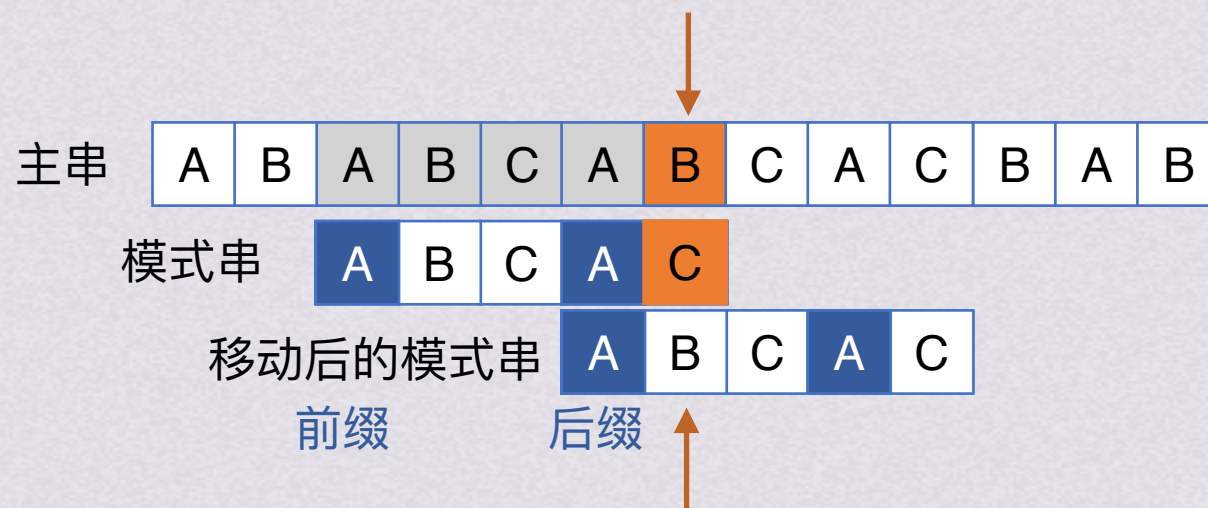
2.KMP算法 next数组





串的模式匹配算法

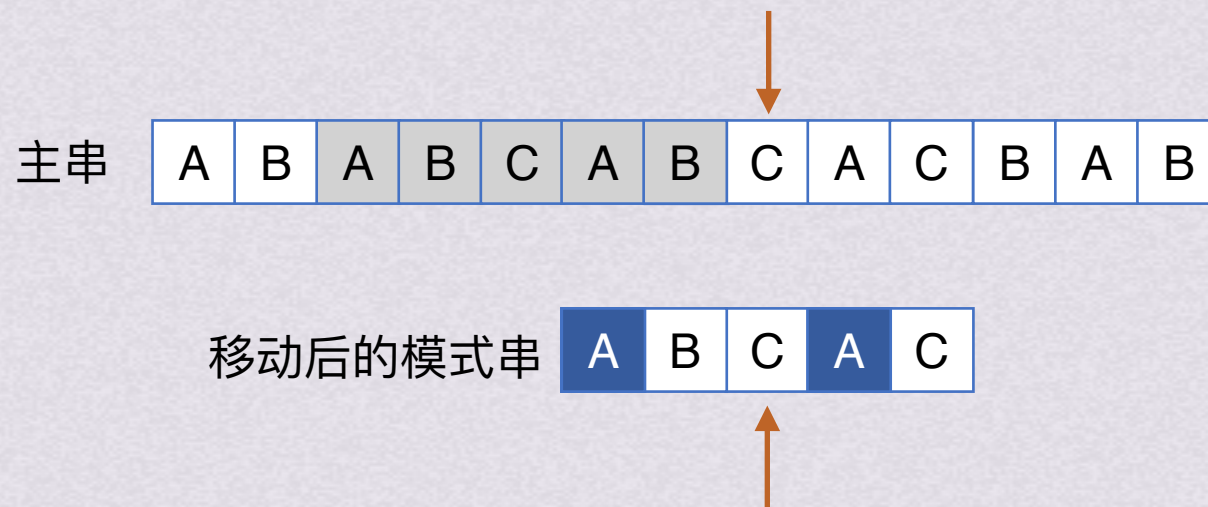
2.KMP算法 next数组





串的模式匹配算法

2.KMP算法 next数组

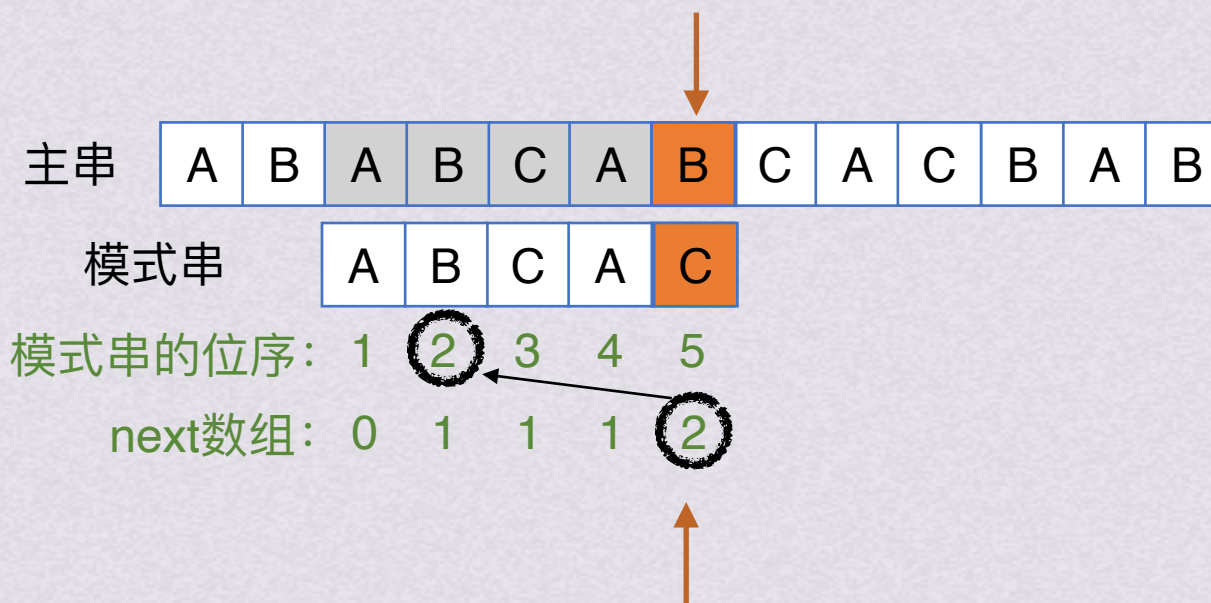




串的模式匹配算法

2.KMP算法 next数组

回到刚才的情况



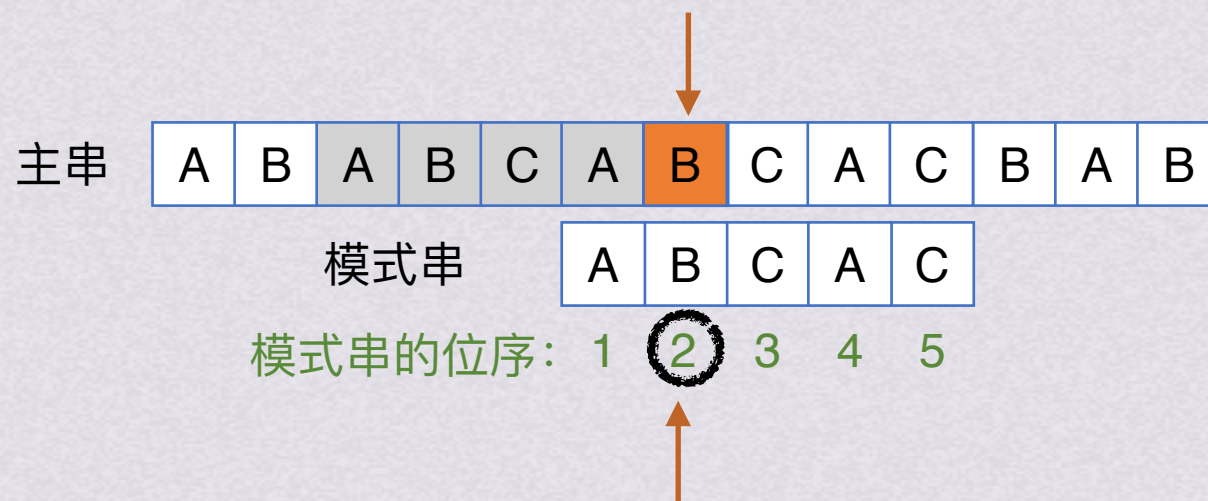
Next数组作用：当模式串数组下标为 i 的位发生失配时，模式串数组下标为 $\text{next}[i]-1$ 的位重新与主串当前位置进行比较



串的模式匹配算法

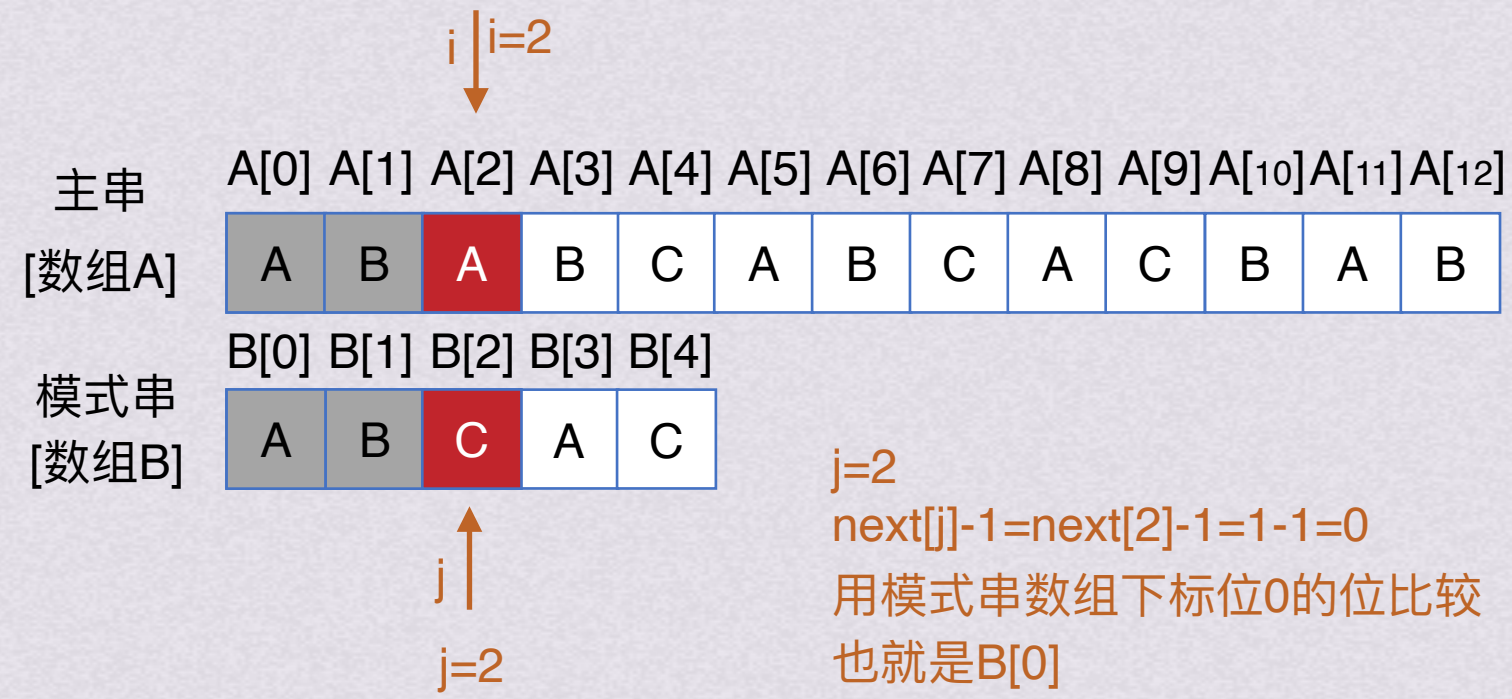
2.KMP算法 next数组

回到刚才的情况



Next数组作用：当模式串数组下标为 i 的位发生失配时，模式串数组下标为 $\text{next}[i]-1$ 的位重新与主串当前位置进行比较

串的模式匹配算法2.KMP算法 next数组



下标	0	1	2	3	4
位序	1	2	3	4	5
模式串	A	B	C	A	C
next	0	1	1	1	2

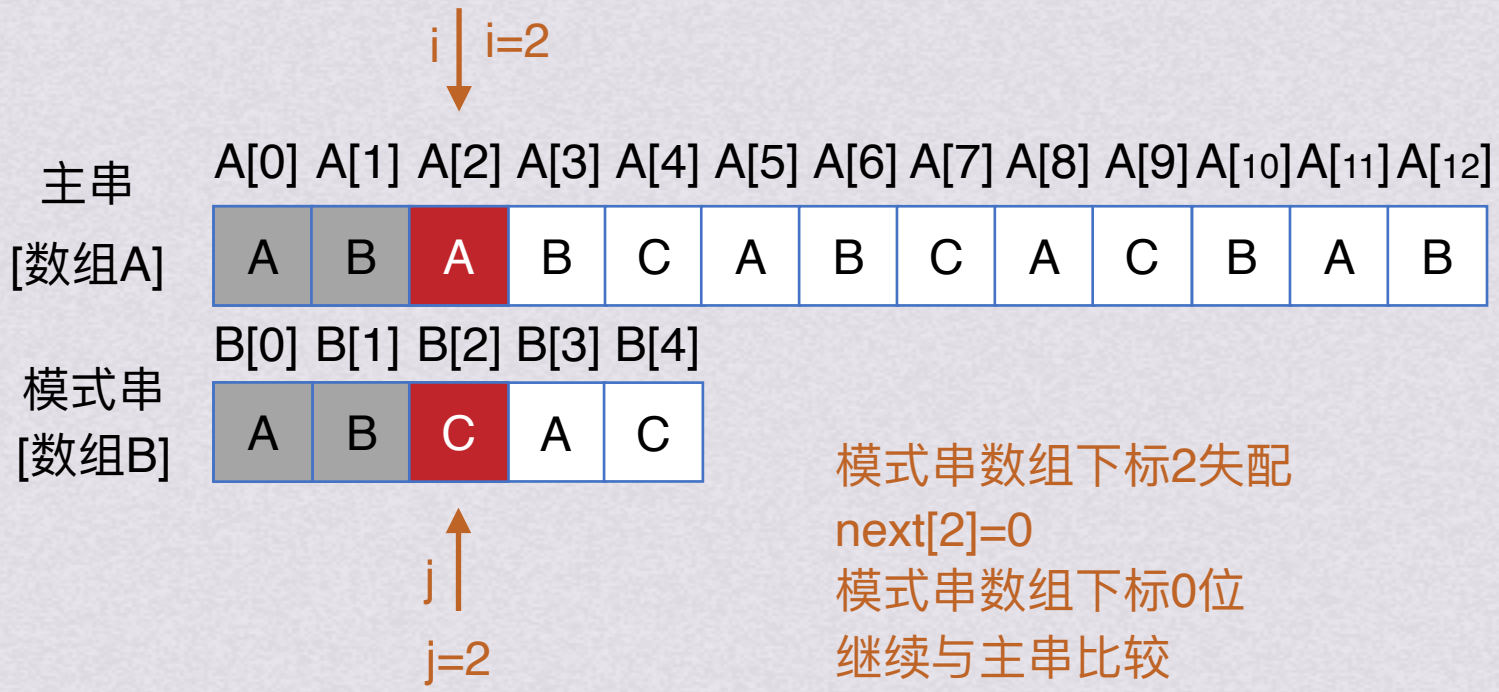
假设next数组从0开始

当模式串中的第i位发生失配时，模式串的第next[i-1]位将重新与主串当前位置进行比较



Next数组作用：当模式串数组下标为i的位发生失配时，模式串数组下标为next[i]-1的位重新与主串当前位置进行比较

串的模式匹配算法2.KMP算法 next数组



next数组从0开始

Next数组作用：当模式串数组下标为i的位发生失配时，模式串数组下标为 $next[i]-1$ 的位重新与主串当前位置进行比较

等价于

next数组从-1开始

当模式串数组下标为i的位发生失配时，模式串数组下标为 $next[i]$ 的位重新与主串当前位置进行比较

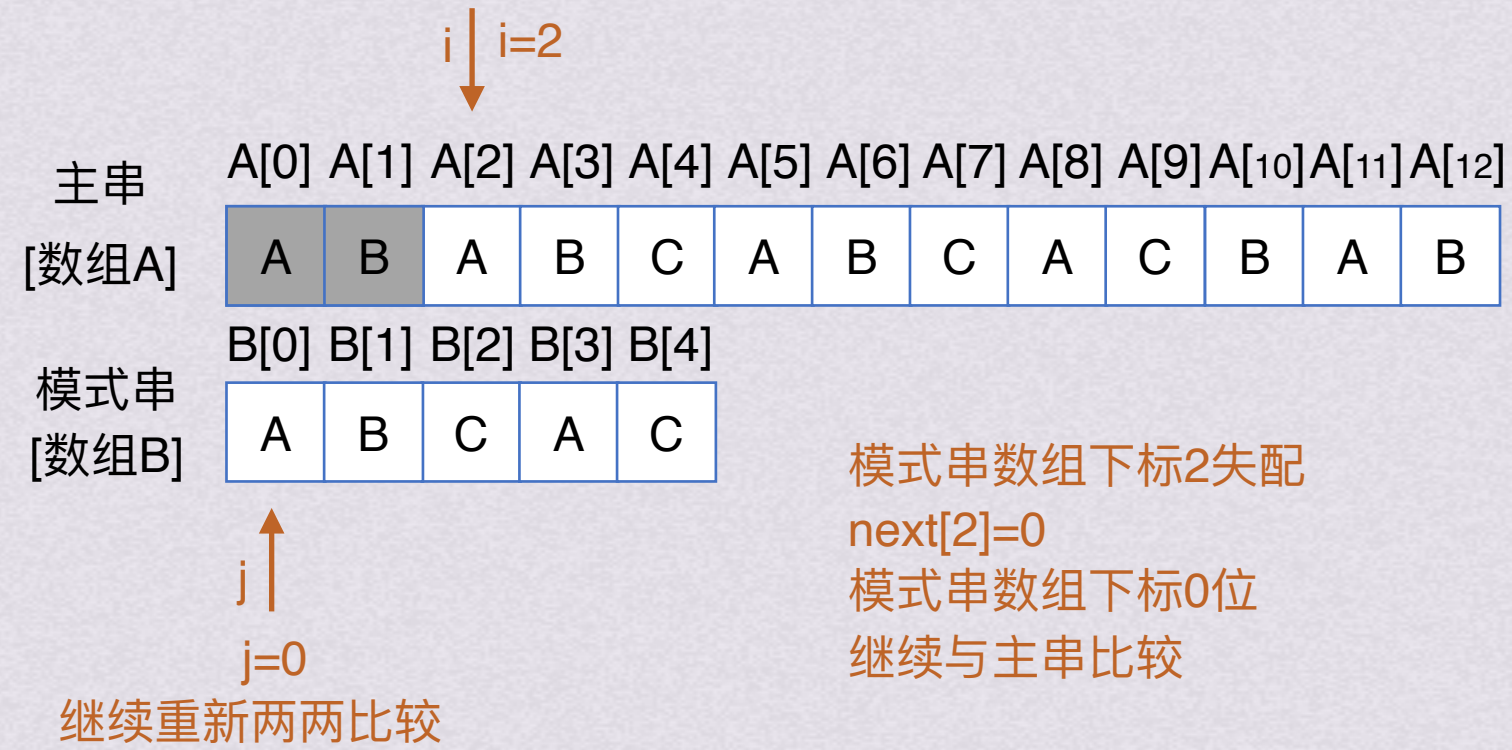
位序	1	2	3	4	5
模式串	A	B	C	A	C
next	0	1	1	1	2

假设next数组从0开始

位序	1	2	3	4	5
模式串	A	B	C	A	C
next	-1	0	0	0	1

将之前的next数组值全部-1
假设next数组从-1开始

串的模式匹配算法2.KMP算法 next数组



next数组从0开始

Next数组作用：当模式串数组下标为i的位发生失配时，模式串数组下标为 $next[i]-1$ 的位重新与主串当前位置进行比较

等价于

next数组从-1开始

当模式串数组下标为i的位发生失配时，模式串数组下标为 $next[i]$ 的位重新与主串当前位置进行比较

位序	1	2	3	4	5
模式串	A	B	C	A	C
next	0	1	1	1	2

假设next数组从0开始

位序	1	2	3	4	5
模式串	A	B	C	A	C
next	-1	0	0	0	1

将之前的next数组值全部-1
假设next数组从-1开始

串的模式匹配算法2.KMP算法 next数组



模式串数组下标4失配
 $next[4]=1$
模式串数组下标1位
继续与主串比较

继续重新两两比较

next数组从0开始

Next数组作用：当模式串数组下标为*i*的位发生失配时，模式串数组下标为 $next[i]-1$ 的位重新与主串当前位置进行比较

等价于

next数组从-1开始

当模式串数组下标为*i*的位发生失配时，模式串数组下标为 $next[i]$ 的位重新与主串当前位置进行比较

位序	1	2	3	4	5
模式串	A	B	C	A	C
next	0	1	1	1	2

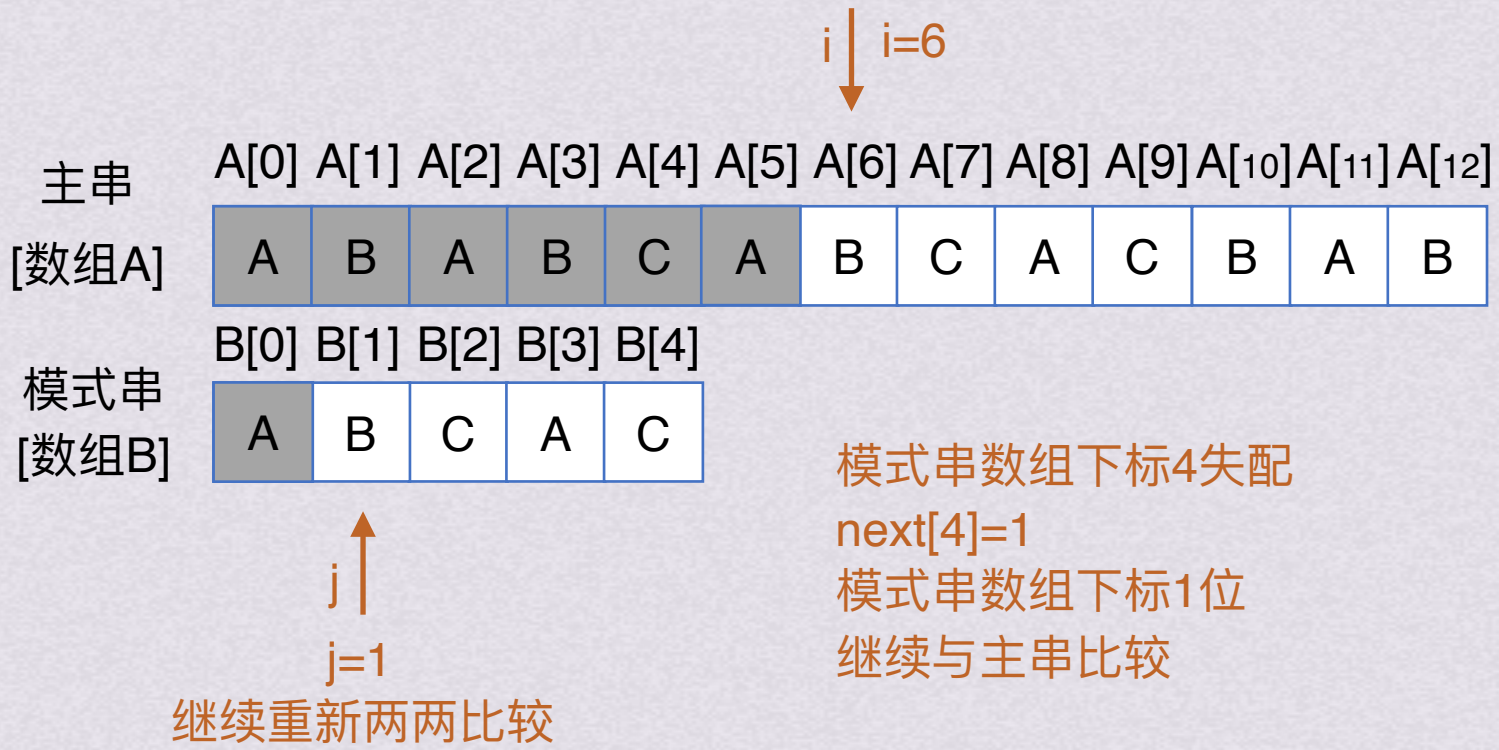
假设next数组从0开始

位序	1	2	3	4	5
模式串	A	B	C	A	C
next	-1	0	0	0	1

将之前的next数组值全部-1

假设next数组从-1开始

串的模式匹配算法2.KMP算法 next数组



模式串数组下标4失配
 $next[4]=1$
模式串数组下标1位
继续与主串比较

next数组从0开始

Next数组作用：当模式串数组下标为*i*的位发生失配时，模式串数组下标为 $next[i]-1$ 的位重新与主串当前位置进行比较

等价于

next数组从-1开始

当模式串数组下标为*i*的位发生失配时，模式串数组下标为 $next[i]$ 的位重新与主串当前位置进行比较

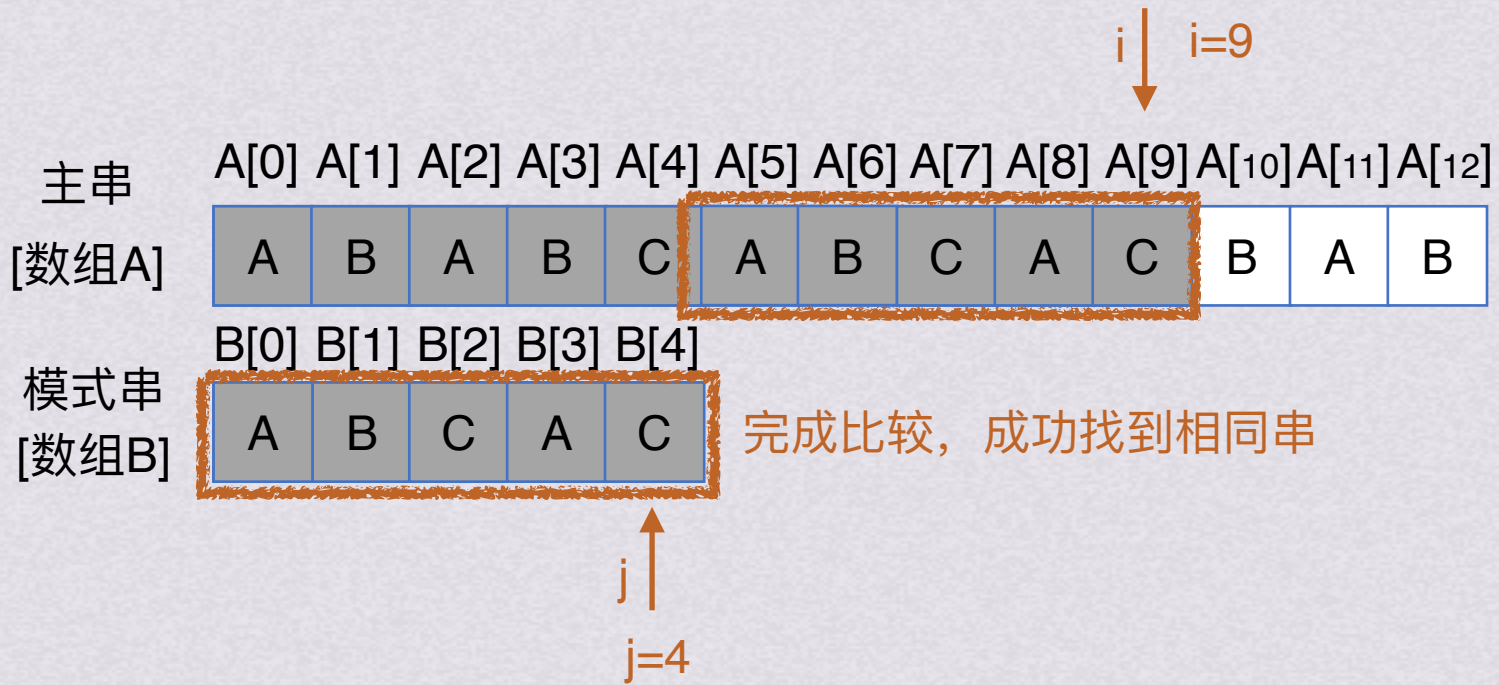
位序	1	2	3	4	5
模式串	A	B	C	A	C
next	0	1	1	1	2

假设next数组从0开始

位序	1	2	3	4	5
模式串	A	B	C	A	C
next	-1	0	0	0	1

将之前的next数组值全部-1
假设next数组从-1开始

串的模式匹配算法2.KMP算法 next数组



继续重新两两比较

next数组从0开始

Next数组作用：当模式串数组下标为i的位发生失配时，模式串数组下标为next[i]-1的位重新与主串当前位置进行比较

等价于

next数组从-1开始

当模式串数组下标为i的位发生失配时，模式串数组下标为next[i]的位重新与主串当前位置进行比较

位序	1	2	3	4	5
模式串	A	B	C	A	C
next	0	1	1	1	2

假设next数组从0开始

位序	1	2	3	4	5
模式串	A	B	C	A	C
next	-1	0	0	0	1

将之前的next数组值全部-1

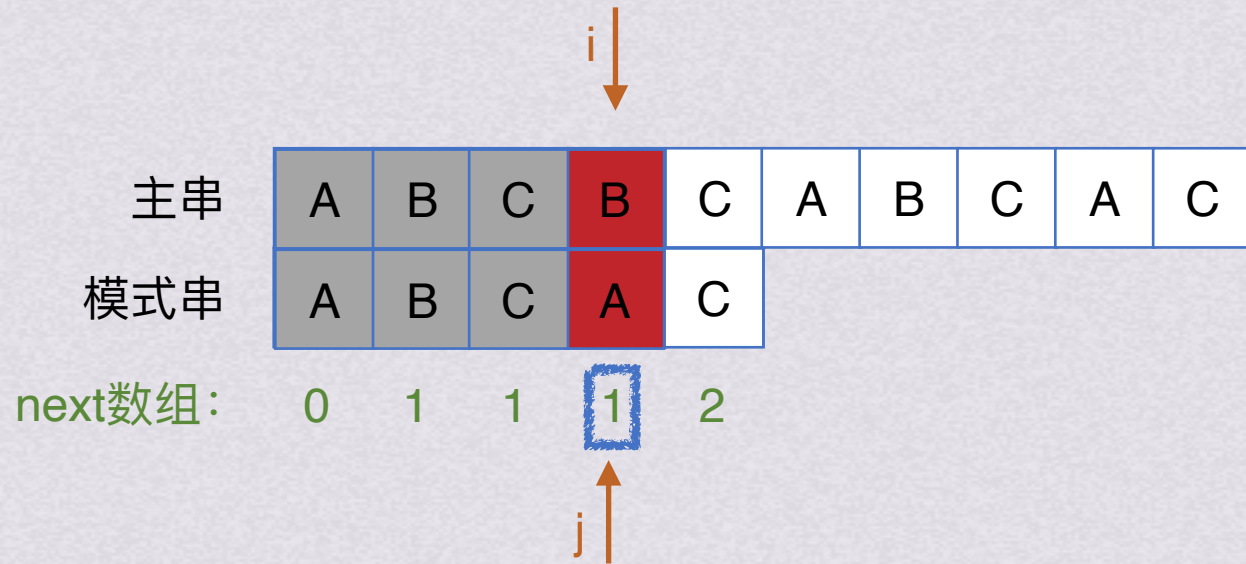
假设next数组从-1开始



串的模式匹配算法

2.KMP算法 `nextval`数组

观察这一对主串和模式串的比较

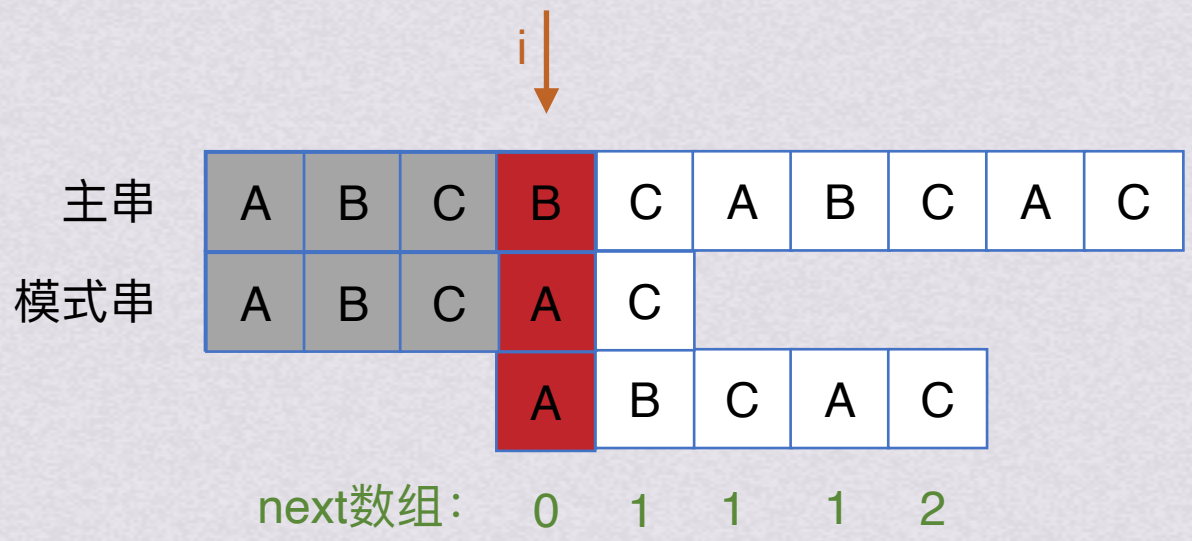


主串的B与模式串的A比较
因为主串B和模式串的A并不相同
因此需要让模式串“前进”后再次比较

串的模式匹配算法

2.KMP算法 `nextval`数组

观察这一对主串和模式串的比较



主串的B与模式串的A比较
因为主串B和模式串的A并不相同
因此需要让模式串“前进”后再次比较

然而模式串“前进”后，仍然是一个A和主串的B进行比较
必定会再次比较出不相符的结果
所以这一次比较是没有意义的

为了跳过这种没有意义的比较，伟大的人类发明了`nextval`数组！





串的模式匹配算法2.KMP算法 (one more thing)

nextval数组的求法

nextval数组的求法：

- 依次观察next数组每一位。
- 对比【本位字符】和【本位字符的next数组值对应的字符】。
- 如果二者相同，则nextval值改为next数组值对应的字符的 nextval 数组值。
- 如果二者不同，nextval值和本位的next数组值相同。

来个例子：

数组位序	1	2	3	4	5
	A	B	C	A	C
由前文可求得next数组为	0	1	1	1	2
nextval	0				

仍然是0
跳过不管



串的模式匹配算法2.KMP算法 (one more thing)

nextval数组的求法

nextval数组的求法：

- 依次观察next数组每一位。
- 对比【本位字符】和【本位字符的next数组值对应的字符】。
- 如果二者相同，则nextval值改为next数组值对应的字符的 nextval 数组值。
- 如果二者不同，nextval值和本位的next数组值相同。

来个例子：

数组位序	1	2	3	4	5
	A	B	C	A	C
由前文可求得next数组为	0	1	1	1	2
nextval	0	1			

检查1号位置是不是B
1号不是B，所以沿用next



串的模式匹配算法2.KMP算法 (one more thing)

nextval数组的求法

nextval数组的求法：

- 依次观察next数组每一位。
- 对比【本位字符】和【本位字符的next数组值对应的字符】。
- 如果二者相同，则nextval值改为next数组值对应的字符的 nextval 数组值。
- 如果二者不同，nextval值和本位的next数组值相同。

来个例子：

数组位序	1	2	3	4	5
	A	B	C	A	C
由前文可求得next数组为	0	1	1	1	2
nextval	0	1	1		



检查1号位置是不是C
1号不是C，所以沿用next



串的模式匹配算法2.KMP算法 (one more thing)

nextval数组的求法

nextval数组的求法：

- 依次观察next数组每一位。
- 对比【本位字符】和【本位字符的next数组值对应的字符】。
- 如果二者相同，则nextval值改为next数组值对应的字符的 nextval 数组值。
- 如果二者不同，nextval值和本位的next数组值相同。

来个例子：

数组位序	1	2	3	4	5
	A	B	C	A	C
由前文可求得next数组为	0	1	1	1	2
nextval	0	1	1	0	



检查1号位置是不是A

1号确实是A，使用1号的next (0)



串的模式匹配算法2.KMP算法 (one more thing)

nextval数组的求法

nextval数组的求法：

- 依次观察next数组每一位。
- 对比【本位字符】和【本位字符的next数组值对应的字符】。
- 如果二者相同，则nextval值改为next数组值对应的字符的 nextval 数组值。
- 如果二者不同，nextval值和本位的next数组值相同。

来个例子：

数组位序	1	2	3	4	5
	A	B	C	A	C
由前文可求得next数组为	0	1	1	1	2
nextval	0	1	1	0	2



检查2号位置是不是C
不是，所以沿用上面next



KMP算法的真正核心和高频易考点：计算next数组和nextval数组

一定要亲自动手算
一定要亲自动手算
要亲自动手算
亲自动手算
自动手算
动手算
手算
算